Bic-DC: Scalable and Privacy-Preserving Multi-Party Collaborative Data Cleaning

Yuxin Xi, Yu Guo, *Senior Member, IEEE*, Chen Zhang, Purong Zhang, Hai Liu, and Xiaohua Jia, *Fellow, IEEE*

***Abstract*—Data cleaning is a foundational yet resource-intensive stage in the data science pipeline. In multi-source settings, collaborative cleaning is essential for resolving cross-source label inconsistencies and improving model reliability. However, existing privacy-preserving solutions mainly support two-party cleaning, while their direct multi-party extensions incur prohibitive pairwise overhead. Moreover, reducing collaborative cleaning to multi-party intersection testing reveals more information than necessary: it exposes all common records, although only records with conflicting labels are relevant to cleaning. We propose Bic-DC, a scalable and privacy-preserving protocol for multi-party collaborative data cleaning. The key idea is to transform multi-party mislabeled-record detection into a bicentric resolution task. Non-bicentric parties locally encode their indexed record–label pairs into oblivious key–value store (OKVS) structures and submit compact encodings to two bicentric parties, thereby avoiding quadratic pairwise interactions. To reveal only records with conflicting labels while hiding consistent overlaps and non-overlapping records, Bic-DC integrates a prefix-flipping technique that converts label inconsistency detection into direct matching over PRF-masked record–label tokens. We formally prove security in the semi-honest model and provide a detailed complexity analysis. Experiments show that Bic-DC reduces communication by up to $3.4\times$ and runtime by up to $5.3\times$ compared with the best-performing pairwise data-cleaning baseline, while scaling to multi-party settings.**

***Index Terms*—Multi-party data cleaning, privacy-preserving, oblivious key-value store, minimal disclosure, scalability.**

I. Introduction

DATA cleaning is widely recognized as a critical yet resource-intensive stage in the data science pipeline [1], [2], [3], [4], [5]. In multi-source settings, collaborative cleaning is needed to resolve label inconsistencies across datasets and improve AI model robustness. As illustrated in Fig. 1, different data sources may assign conflicting labels to identical records. Such inconsistencies arise in domains such as radiotherapy, where organ-at-risk annotations may be inconsistent [6], and finance, where identical transactions may receive divergent labels [7]. Real-world datasets are often held by mutually untrusted parties, and directly exchanging records or labels raises serious privacy concerns, calling for privacy-preserving collaborative cleaning protocols that detect mislabeled records without exposing raw datasets.

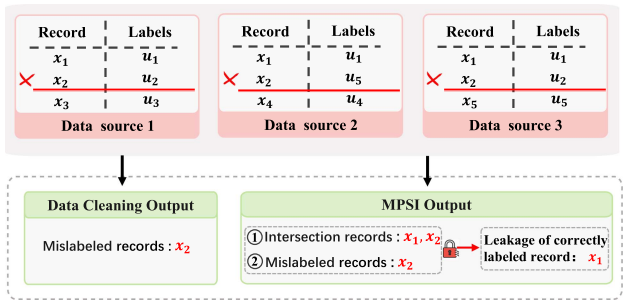


Fig. 1. Collaborative data cleaning scenario and over-disclosure in MPSI.

Recently, Blass et al. [8] proposed the first privacy-preserving collaborative data cleaning protocol by formulating two-party label-mismatch detection as a variant of private set intersection (PSI) [9], [10], [11], [12], [13], [14], [15], [16], [17], [18]. However, this pairwise design does not scale to multi-party collaboration: a direct extension requires quadratic interactions and redundant label comparisons.

A natural alternative is to first identify records shared by all parties using multi-party private set intersection(MPSI) [19], [20], [21], [22], [23], [24], and then check whether their labels are consistent. This intersection-first strategy is efficient for finding common records, but it reveals more information than required for data cleaning. As illustrated in Fig. 1, both x_1 and x_2 are exposed during intersection testing, although only x_2 has inconsistent labels. The consistently labeled record x_1 is irrelevant to the cleaning output, yet still becomes visible, violating the goal of avoiding unnecessary data disclosure. Therefore, existing techniques fail to simultaneously provide scalability and task-specific privacy for multi-party collaborative data cleaning.

Motivated by these limitations, we ask the following question:

Received 28 November 2025; revised 1 June 2026; accepted 24 June 2026. Date of publication 29 June 2026; date of current version 1 September 2026. This work was supported in part by the National Key R&D Program of China under Grant 2022ZD0115901, in part by the Big Data Intelligence Centre at The Hang Seng University of Hong Kong, Beijing Key Laboratory of Artificial Intelligence for Education, Engineering Research Center of Intelligent Technology and Educational Application, Ministry of Education, in part by the Research Grants Council of Hong Kong under FDS Grant UGC/FDS14/E02/25, in part by the Hong Kong Research Grants Council under Grant RFS2425-1S01 and Grant R1012-21, in part by the National Natural Science Foundation of China under Grant 62102035, in part by Ant Group through the CCF-Ant Research Fund under Grant CCF-AFSG RF20240403, and in part by the Open Research Fund of The State Key Laboratory of Blockchain and Data Security, Zhejiang University under Grant A2529. *(Corresponding author: Yu Guo.)*

Yuxin Xi, Yu Guo, and Purong Zhang are with the School of Artificial Intelligence, Beijing Normal University, Beijing 100088, China (e-mail: xiyx@mail.bnu.edu.cn; yuguo@bnu.edu.cn; zpr@mail.bnu.edu.cn).

Chen Zhang and Hai Liu are with the Department of Computer Science, Hang Seng University of Hong Kong, Siu Lek Yuen Hong Kong (e-mail: czhang@hsu.edu.hk; hliu@hsu.edu.hk).

Xiaohua Jia is with the Department of Computer Science, City University of Hong Kong, Kowloon Hong Kong (e-mail: csjia@cityu.edu.hk).

Digital Object Identifier 10.1109/TDSC.2026.3707938

Can we design a scalable protocol for privacy-preserving multi-party data cleaning that identifies only mislabeled records without exposing irrelevant common records?

In this paper, we present Bic-DC, a scalable protocol tailored to privacy-preserving multi-party data cleaning. Bic-DC identifies mislabeled records, i.e., identical records assigned different labels, while avoiding the disclosure of consistently labeled overlaps and non-overlapping records. To address the scalability challenge, we introduce a bicentric detection scheme that transforms the multi-party inconsistency-detection task into an efficient two-party detection instance. Specifically, non-bicentric parties locally mask their records and labels using coordinator-distributed pseudorandom function (PRF) keys, embed the masked values into oblivious key–value store (OKVS) structures, and forward only these compact encodings to the bicentric parties. The bicentric parties then complete the detection solely from the received encodings without further interaction among non-bicentric parties. This architecture decouples local preprocessing from bicentric resolution, enabling constant-round execution while avoiding quadratic pairwise interactions. To avoid over-disclosure and prevent intersection leakage, we integrate a lightweight prefix-flipping technique into the bicentric detection scheme, reformulating label-inconsistency detection as a direct matching task that reveals only mislabeled records while hiding irrelevant common records. The main contributions of this paper are summarized as follows:

- We formalize the privacy-preserving multi-party collaborative data cleaning problem and propose Bic-DC, the first highly scalable data cleaning protocol that reveals only mislabeled records while hiding consistent and non-overlapping records.
- We design a bicentric detection scheme that transforms the multi-party data cleaning task into a two-party label-inconsistency detection problem, achieving high efficiency and scalability for large numbers of participants while avoiding quadratic pairwise interactions.
- We provide a formal security proof of Bic-DC and conduct extensive experiments to evaluate its performance. The results show that Bic-DC achieves up to $3.4\times$ lower communication and $5.3\times$ lower runtime than the best-performing pairwise data-cleaning baseline, while its matching core remains competitive with recent MPSI-style protocols.

II. Related Work

Data Cleaning: Most existing data-cleaning studies focus on single-source settings [25], [26], [27]. Early methods relied on integrity constraints or statistical inference to repair errors, while later approaches incorporated machine learning, such as ActiveClean [28] and CleanML [29]. Recent studies further show that label errors are particularly harmful to model quality, motivating label-cleaning techniques such as TARS [30], [31], [32], [33]. Beyond single-source cleaning, collaborative cleaning has also been explored in distributed environments, e.g., optimizing edge-side cleaning of numerical and structural errors [34]. However, existing efforts mainly target local correction or system-level efficiency, rather than privacy-preserving detection of semantic label inconsistencies across mutually untrusted data sources. Thus, secure multi-party solutions for resolving cross-source label conflicts remain largely unexplored.

Privacy-Preserving Collaborative Data Cleaning: Blass et al. [8] proposed the first protocol for privacy-preserving collaborative data cleaning, extending PSI to detect records with identical values but inconsistent labels. However, their design is confined to the two-party setting and does not scale efficiently to larger collaborations. Naive extensions to multiple parties incur prohibitive computation and communication costs due to repeated pairwise executions. MPSI protocols [35], [36], [37] naturally extend PSI to more than two parties, but their intersection-first functionality reveals all common records, whereas collaborative cleaning only needs records with inconsistent labels.

Kolesnikov et al. [36] proposed an efficient MPSI protocol using oblivious transfer (OT) extension and an oblivious programmable PRF (OPPRF). The protocol requires pairwise OPPRF executions between parties, leading to substantial communication and multiple interaction rounds. Kavousi et al. [38] improved this line by adopting a star topology and garbled bloom filter (GBF) encoding for compact aggregation. However, its star topology still concentrates computation at a designated leader, and its probabilistic encoding is not tailored to mislabeled-record-only disclosure. Zhang et al. [18] proposed a maliciously secure MPSI protocol under the assumption that two designated central parties do not collude. It follows an offline–online design, but its preprocessing and computation overhead may limit scalability in large-scale settings. Nevo et al. [16] enhanced collusion tolerance through a ZeroXOR mechanism built on OPPRF, supporting up to $t < n$ corrupted parties. However, its cost grows with the set size and corruption threshold, which can be expensive in large deployments. Gao et al. [35] proposed an efficient semi-honest MPSI protocol based on OKVS sharing and reconstruction using symmetric-key primitives, thereby reducing the overhead of public-key operations.

Despite these advances, existing MPSI protocols remain unsuitable for collaborative data-cleaning tasks because they expose full intersections rather than directly identifying label inconsistencies. Our Bic-DC protocol addresses this gap through an OKVS-based bicentric architecture that avoids quadratic pairwise interactions while preserving task-specific privacy for multi-party data cleaning.

III. Preliminaries

In this section, we introduce the main building blocks used in Bic-DC. Frequently used notation is summarized in Table I.

A. Oblivious Key-Value Store

An oblivious key–value store (OKVS) [39] encodes a set of key–value pairs into a compact object that supports efficient decoding for queried keys while hiding the encoded key set. An OKVS is defined over a key domain $\mathcal{K}$ and a value domain $\mathcal{V}$, and provides two algorithms:

TABLE I
IMPORTANT NOTATIONS

Notation	Description
$P = \{P_1, \ldots, P_n\}$	Set of participating parties
n	Number of parties
S_i	Private database of P_i
m	Number of local records
(x_j^i, u_j^i)	j-th record–label pair of P_i
X_i	Record set of P_i
X	Union of all parties' record sets
$\ell_i(x)$	Label assigned to x by P_i
C	Mislabeled-record output set
λ_u	Label-encoding length
k_i	PRF key shared by P_1 and P_i
D_i	Record-indexed OKVS encoding of P_i
$\widetilde{D}_i$	Label-indexed OKVS encoding of P_i
b_j^i	Decoded masked record value
$\widetilde{b}_j^i$	Decoded masked label value
Z_j^i	Prefix-token set for the j-th entry
M	Matched prefix-token set
μ	Bicentric matching transcript leakage

- $\mathsf{Encode}(\{(r_i, v_i)\}_{i\in[m]})$: Given a set of key–value pairs $\{(r_i, v_i)\}_{i\in[m]} \subseteq \mathcal{K} \times \mathcal{V}$ with distinct keys, output an encoded object D or $\perp$ if encoding fails.
- $\mathsf{Decode}(D, r)$: Given an encoded object D and a key $r \in \mathcal{K}$, output the associated value in $\mathcal{V}$.

Correctness: An OKVS is *correct* if, for any set of distinct key–value pairs $L = \{(r_i, v_i)\}_{i\in[m]}$ and any successful encoding $D = \mathsf{Encode}(L) \neq \perp$, it holds that

$$\forall (r_i, v_i) \in L, \quad \mathsf{Decode}(D, r_i) = v_i.$$

Obliviousness: An OKVS is *oblivious* if its encoding hides the underlying key set when the associated values are independently random. Specifically, let $R_0 = \{r_i^0\}_{i\in[m]}$ and $R_1 = \{r_i^1\}_{i\in[m]}$ be any two key sets of equal size. Consider the following two distributions:

$$\mathcal{D}_0 = \left\{ D \mid v_i \leftarrow \mathcal{V},\ D \leftarrow \mathsf{Encode}(\{(r_i^0, v_i)\}_{i\in[m]}) \right\},$$
$$\mathcal{D}_1 = \left\{ D \mid v_i \leftarrow \mathcal{V},\ D \leftarrow \mathsf{Encode}(\{(r_i^1, v_i)\}_{i\in[m]}) \right\}.$$

For any probabilistic polynomial-time adversary $\mathcal{A}$, it holds that $|\Pr[\mathcal{A}(D) = 1 : D \leftarrow \mathcal{D}_0] - \Pr[\mathcal{A}(D) = 1 : D \leftarrow \mathcal{D}_1]| \leq \mathsf{negl}(\kappa)$, where κ is the security parameter. Thus, the encoding leaks no information about the encoded keys beyond the set size, provided that the values are independently random. In Bic-DC, the values stored in OKVS are PRF outputs or XORs of PRF outputs; under the PRF security assumption, these values are computationally indistinguishable from random values and therefore satisfy the randomness requirement for OKVS obliviousness.

B. Prefix-Flipping for Label-Mismatch Detection

As illustrated in Fig. 2, we use a prefix-flipping technique to transform label inequality testing into a set-matching problem. Given a bit string u of length λ_u, let $\mathsf{Prefix}_\ell(u)$ denote its first ℓ bits, where $\ell \in [\lambda_u]$. For a non-empty bit string s, let $\mathsf{FlipLast}(s)$ denote the string obtained by flipping the last bit of s.

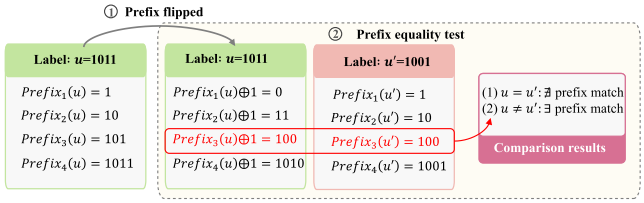


Fig. 2. Prefix-flipping for secure label comparison.

For another bit string u', we compare the flipped prefixes of u with the unmodified prefixes of u'. This comparison satisfies the following properties:

- If $u = u'$, then no flipped prefix of u matches the corresponding unmodified prefix of u'.
- If $u \neq u'$, then there exists at least one index $i \in [\ell]$ such that$\mathsf{FlipLast}(\mathsf{Prefix}_i(u)) = \mathsf{Prefix}_i(u')$.

For example, when $u = 1011$ and $u' = 1001$, the first differing bit appears at $i = 3$, and we have $\mathsf{FlipLast}(\mathsf{Prefix}_3(u)) = 100 = \mathsf{Prefix}_3(u')$. Thus, prefix flipping converts label-mismatch detection into a prefix-set matching problem. In Bic-DC, this technique is applied to PRF-masked label encodings and combined with masked record encodings, so that a match indicates an identical record with inconsistent labels without exposing the original labels.

C. Pseudorandom Function (PRF)

A pseudorandom function (PRF) [40] is a keyed function $\mathsf{PRF}_k : \{0,1\}^* \to \{0,1\}^\lambda$ such that, for a uniformly chosen key k, the function $\mathsf{PRF}_k(\cdot)$ is computationally indistinguishable from a truly random function with the same domain and range. More formally, no probabilistic polynomial-time adversary can, with non-negligible advantage, distinguish between oracle access to $\mathsf{PRF}_k(\cdot)$ and to a uniformly random function.

IV. System Model and Problem Definition

A. Problem Definition

We consider a collaborative setting with n mutually untrusted parties $P = \{P_1, P_2, \ldots, P_n\}$, where each party P_i holds a private database $S_i = \{(x_j^i, u_j^i)\}_{j=1}^m$ with m entries, as illustrated in Fig. 3. Each entry consists of a record x_j^i (e.g., a record identifier) and its associated label u_j^i. Let $X_i = \{x_j^i\}_{j=1}^m$ denote the record set of party P_i, and let $X = \bigcup_{i\in[n]} X_i$ denote the set of all records appearing in at least one party's database. For $x \in X_i$, let $\ell_i(x)$ denote the label associated with x in S_i. Each record $x \in X$ falls into one of the following categories:

1) *Mislabeled record:* $x \in \bigcap_{i\in[n]} X_i$, and there exist two parties P_p and P_q such that $\ell_p(x) \neq \ell_q(x)$.
2) *Consistent record:* $x \in \bigcap_{i\in[n]} X_i$, and all parties assign the same label to x, i.e., $\ell_1(x) = \ell_2(x) = \cdots = \ell_n(x)$.
3) *Non-overlapping record:* $x \in \bigcup_{i\in[n]} X_i$ but $x \notin \bigcap_{i\in[n]} X_i$, i.e., x is absent from at least one party's database.

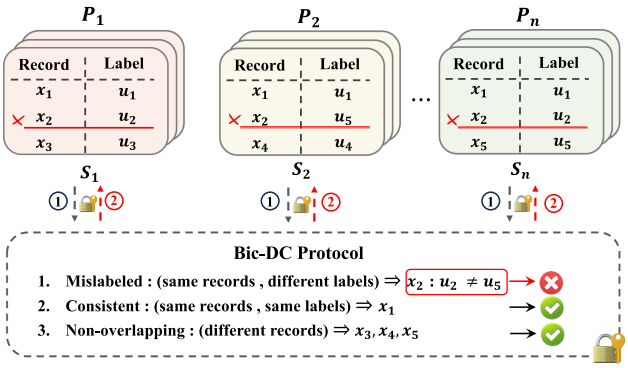


Fig. 3. System model of Bic-DC: multiple parties detect mislabeled records without revealing other data.

Ideal functionality and leakage: We define an ideal functionality $\mathcal{F}_{\mathsf{MCDC}}$ that, given private databases $S_1, \ldots, S_n$, outputs only the mislabeled-record set

$$C = \left\{ x \in \bigcap_{i \in [n]} X_i \mid \exists p, q \in [n],\ \ell_p(x) \neq \ell_q(x) \right\}$$

to the authorized parties. To explicitly capture bicentric-resolution leakage, we define

$$\mathcal{L}_{\text{Bic-DC}} = (n, m, \lambda_u, \mathsf{size}_{\mathsf{msg}}, \mu, \mathsf{abort}),$$

where λ_u is the label-encoding length, $\mathsf{size}_{\mathsf{msg}}$ denotes fixed transmitted-message sizes, μ denotes the bicentric matching transcript visible to the bicentric parties, and abort denotes public abort behavior. Specifically, μ includes the match/no-match outcomes over PRF-masked record–label tokens and the number of matched tokens before duplicate removal; token-set sizes are determined by public parameters and captured by $\mathsf{size}_{\mathsf{msg}}$. Security is defined as revealing nothing beyond the prescribed output C and the explicit leakage profile $\mathcal{L}_{\text{Bic-DC}}$.

B. Threat Model

We analyze the security of Bic-DC in the semi-honest adversarial model within the real/ideal simulation framework [41], [42]. All parties are assumed to follow the prescribed protocol, but corrupted parties may attempt to infer additional information from their local views, including their inputs, randomness, received messages, intermediate states, and outputs.

We consider static corruptions under a non-collusion assumption for the coordinator-assisted bicentric architecture. Specifically, we assume that the coordinator P_1 does not collude with the bicentric parties, since P_1 participates in PRF-key setup whereas the bicentric parties perform the final resolution over encoded values. Under this assumption, Bic-DC aims to reveal only records identified as mislabeled according to the prescribed output policy, while hiding consistently labeled overlaps and non-overlapping records.

The security goal is formalized by showing that, for every allowed semi-honest corruption pattern, there exists a probabilistic polynomial-time simulator that can generate a view computationally indistinguishable from the real execution using only the corrupted parties' inputs and outputs. The formal proof is provided in Section VI.

The current construction uses a coordinator-assisted PRF-key setup for simplicity. To reduce trust concentration in P_1, this setup can be instantiated by a lightweight decentralized key-generation procedure, such as commitment-based coin tossing or distributed key generation (DKG)-inspired key establishment [43], [44]. Each PRF key is jointly derived from randomness contributed by P_1 and the corresponding party, preventing any single party from unilaterally choosing or biasing the key. This affects only the one-time setup phase and does not change the asymptotic online complexity of Bic-DC.

V. The Bic-DC Design

A. Design Rationale

The fundamental challenge of collaborative data cleaning among mutually untrusted parties lies in designing a secure mechanism for privacy-preserving record matching. The task requires identifying identical records across datasets and subsequently comparing their associated labels to detect inconsistencies. In the multi-party setting, however, each participant protects its data under independent local encodings, making it particularly difficult to achieve matching that is both consistent and privacy-preserving.

To address this challenge, a promising technique is OKVS [45], [46]. At a high level, OKVS compactly encodes key–value pairs into a structure that supports efficient decoding of selected keys, such as records or labels, while keeping the encoded key set hidden. This makes OKVS well suited for transmitting succinct dataset representations in multi-party data cleaning. By processing each record or label key through a PRF, we derive values such that identical inputs across parties map to common pseudorandom tags, e.g., via XOR of PRF outputs, enabling consistent matching while keeping distinct inputs computationally indistinguishable.

However, directly extending OKVS with PRF to multiple parties requires every participant to interact with all others, leading to quadratic communication and coordination costs. The key intuition of Bic-DC is to reduce the n-party detection task to a bicentric resolution task. Each participant encodes its local record–label pairs into compact OKVS objects and submits them once to two designated bicentric parties. Through PRF-masked OKVS decoding, the bicentric parties reconstruct a common masked matching space from these one-time submissions, without exposing raw records or labels beyond the prescribed output and explicit leakage profile. When a record appears across all participants, its decoded OKVS value coincides at both bicentric parties; otherwise, the decoded values are pseudorandom and unrelated. This enables global inconsistency detection: identical records yield the same decoded record value, while their associated label encodings may diverge. As a result, Bic avoids the $O(n^2)$ pairwise interactions required by naive extensions, while keeping the online communication pattern constant-round and relying only on lightweight symmetric primitives in the online phase.

A straightforward bicentric scheme would first derive the intersection and then compare labels, but this reveals all common records, including those with consistent labels, thus causing unnecessary disclosure. To overcome this limitation, we customize the bicentric detection scheme using a prefix-flipping technique (see Section III-B). The key idea is that when two label encodings differ, their respective prefix sets, one with flipped prefixes and the other with unmodified prefixes, yield a non-empty intersection, whereas consistent label encodings remain disjoint. By attaching label-prefix tokens to their corresponding masked record encodings, comparisons are restricted to identical records, and a match occurs only when the corresponding labels disagree. Since all operations are performed on PRF-masked values decoded from OKVS, original records and labels remain hidden beyond the prescribed mislabeled-record output and the explicitly modeled leakage in $\mathcal{L}_{\text{Bic-DC}}$.

B. Decentralized PRF-Key Setup

Before data encoding, Bic-DC establishes the PRF keys used for OKVS-based masking through a lightweight decentralized key-setup procedure. For each non-bicentric party P_i with $2 \leq i \leq n-2$, the coordinator P_1 and P_i jointly derive a PRF key k_i over authenticated channels. Specifically, P_1 samples $a_i \leftarrow \{0,1\}^\kappa$ and sends a commitment $c_i^1 = \mathsf{Com}(a_i; r_i^1)$ to P_i, while P_i samples $b_i \leftarrow \{0,1\}^\kappa$ and sends $c_i^2 = \mathsf{Com}(b_i; r_i^2)$ to P_1. After exchanging the commitments, both parties open them and verify the openings. They then derive $k_i = H(\mathsf{sid} \parallel i \parallel a_i \parallel b_i)$, where sid is the session identifier and H is used as a key-derivation hash function.

This commitment-based procedure prevents unilateral key selection by either party and reduces trust concentration in the coordinator. The derived key k_i is available only to P_1 and the corresponding P_i, preserving the original encoding interface. The setup adds only a one-time cost of two commitments, two openings, and one hash-based derivation per non-bicentric party, resulting in $O(n\kappa)$ bits of communication and $O(n)$ lightweight cryptographic operations. Since this cost is independent of the dataset size m, it is amortized across all records and does not change the asymptotic online complexity of Bic-DC. In practice, sid binds each derived key to a specific cleaning session, and keys can be refreshed across sessions for key separation. This procedure can be viewed as a DKG-inspired pairwise key-establishment step tailored to the PRF-key sharing pattern required by Bic-DC.

C. Multi-Party Data Representation

In this phase, the non-bicentric input parties transform their local datasets into compact, privacy-preserving representations that can be resolved by the bicentric parties. The core primitive is the OKVS, which conceals keys while enabling efficient key–value queries. To bind each encoding to the parties' PRF contributions, we derive deterministic, key-dependent tags using PRFs. For each party P_i, we define the paired dataset as $S_i = \{(x_j^i, u_j^i)\}_{j\in[m]}$, with projections $X_i = \{x_j^i\}_{j\in[m]}$ and $U_i = \{u_j^i\}_{j\in[m]}$. We instantiate two independent OKVS instances per input party: a record-indexed OKVS D_i keyed by X_i and a label-indexed OKVS $\widetilde{D}_i$ keyed by U_i. Although records and labels are encoded in two OKVS structures, their association is preserved by the local index j and is used later when constructing paired record–label tokens. Each input party submits its compact encodings to the designated bicentric parties. This design avoids the $O(n^2)$ pairwise communication of naive n-party protocols, since each input party communicates only once while keeping raw records and labels hidden beyond the explicitly modeled leakage. The detailed procedure is given in Step 1 and Step 2, as illustrated in Fig. 4.

Step 1: Decentralized PRF-key setup and encoding: For each $2 \leq i \leq n-2$, P_1 and P_i jointly establish a PRF key k_i using the decentralized PRF-key setup in Section V-B. For every pair $(x_j^1, u_j^1) \in S_1$, P_1 evaluates all established PRFs on the same input and aggregates the outputs via bitwise XOR:

$$\alpha_j^1 = \bigoplus_{i=2}^{n-2} \mathrm{PRF}_{k_i}(x_j^1), \qquad \beta_j^1 = \bigoplus_{i=2}^{n-2} \mathrm{PRF}_{k_i}(u_j^1).$$

This aggregation ensures that identical elements yield the same aggregated pseudorandom value across the bicentric reconstruction, while distinct elements remain computationally indistinguishable. The aggregated values are then bound to their corresponding inputs and encoded into OKVS structures:

$$D_1 = \mathsf{OKVS.Encode}\big(\{(x_j^1, \alpha_j^1)\}_{j\in[m]}\big),$$
$$\widetilde{D}_1 = \mathsf{OKVS.Encode}\big(\{(u_j^1, \beta_j^1)\}_{j\in[m]}\big).$$

The encoded structures $(D_1, \widetilde{D}_1)$ are transmitted to the designated bicentric party P_n.

Step 2: Multi-party OKVS encoding: Each P_i $(2 \leq i \leq n-2)$ encodes its dataset locally using the PRF key k_i jointly established with P_1. For every pair $(x_j^i, u_j^i) \in S_i$, it computes the PRF outputs and encodes them into two independent OKVS structures:

$$D_i = \mathsf{OKVS.Encode}\big(\{(x_j^i, \mathrm{PRF}_{k_i}(x_j^i))\}_{j\in[m]}\big),$$
$$\widetilde{D}_i = \mathsf{OKVS.Encode}\big(\{(u_j^i, \mathrm{PRF}_{k_i}(u_j^i))\}_{j\in[m]}\big).$$

The encoded structures $(D_i, \widetilde{D}_i)$ are transmitted to the designated bicentric party P_{n-1}. At the end of this phase, P_{n-1} receives $\{(D_i, \widetilde{D}_i)\}_{i=2}^{n-2}$, while P_n receives $(D_1, \widetilde{D}_1)$. These encodings form the compact representation used for subsequent bicentric decoding and mislabeled detection. The overall procedure of this phase is summarized in Algorithm 1.

D. Bicentric Mislabeled Resolution

In this phase, the two bicentric parties detect label inconsistencies on common records while avoiding over-disclosure. Each bicentric party first decodes its local records and labels using the OKVS structures provided by the non-bicentric parties, so that identical inputs yield the same pseudorandom encoding while distinct inputs remain computationally indistinguishable. Directly deriving the record intersection would expose all common records, including consistently labeled ones. To avoid this leakage, we employ the prefix-flipping technique (see Section III-B): P_{n-1} generates complete prefix sets from its

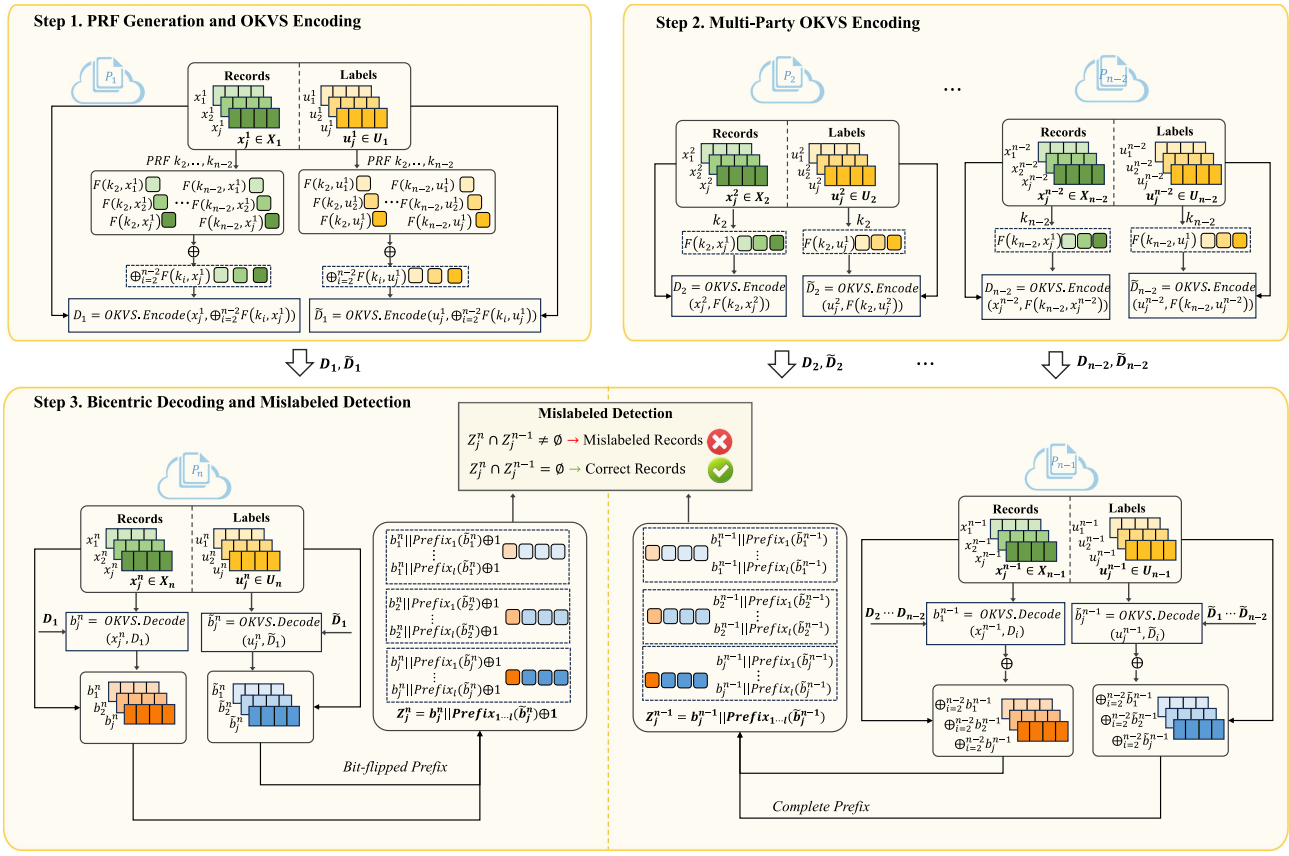


Fig. 4. Detailed design of the Bic-DC protocol.

decoded labels, whereas P_n generates bit-flipped prefix sets. By concatenating each label-prefix token with the corresponding masked record encoding, a match occurs only when the same record is associated with different labels. The detailed procedure is presented in Algorithm 2, and its role in the overall workflow is illustrated as Step 3 in Fig. 4.

Step 3: Bicentric decoding and mislabeled detection: Upon receiving the OKVS structures, the two bicentric parties locally reconstruct masked encodings of their own datasets. For every record–label pair $(x_j^{n-1}, u_j^{n-1}) \in S_{n-1}$, P_{n-1} computes

$$b_j^{n-1} = \bigoplus_{i=2}^{n-2} \mathsf{OKVS.Decode}(D_i, x_j^{n-1}),$$

$$\widetilde{b}_j^{n-1} = \bigoplus_{i=2}^{n-2} \mathsf{OKVS.Decode}(\widetilde{D}_i, u_j^{n-1}).$$

Similarly, for every $(x_j^n, u_j^n) \in S_n$, P_n computes

$$b_j^n = \mathsf{OKVS.Decode}(D_1, x_j^n),$$

$$\widetilde{b}_j^n = \mathsf{OKVS.Decode}(\widetilde{D}_1, u_j^n).$$

By construction, if all parties hold the same record x or label u, then both bicentric parties recover the same aggregated encoding:

$$b_j^{n-1} = b_{j'}^n = \bigoplus_{i=2}^{n-2} \mathsf{PRF}_{k_i}(x),$$

$$\widetilde{b}_j^{n-1} = \widetilde{b}_{j'}^n = \bigoplus_{i=2}^{n-2} \mathsf{PRF}_{k_i}(u).$$

If the inputs differ, the decoded outputs are unrelated pseudorandom values due to PRF pseudorandomness and OKVS obliviousness.

After decoding, both parties convert the reconstructed values into prefix-token sets. Specifically, P_{n-1} generates the complete prefix set

$$Z_j^{n-1} = \left\{ b_j^{n-1} \parallel \mathsf{Prefix}_\ell(\widetilde{b}_j^{n-1}) \right\}_{\ell=1}^{\lambda_u}.$$

while P_n constructs the bit-flipped prefix set

$$Z_j^n = \left\{ b_j^n \parallel \mathsf{FlipLast}\big(\mathsf{Prefix}_\ell(\widetilde{b}_j^n)\big) \right\}_{\ell=1}^{\lambda_u}.$$

After constructing the prefix-token sets, the two bicentric parties perform set matching over their PRF-masked token sets

$$Z^{n-1} = \bigcup_{j\in[m]} Z_j^{n-1}, \qquad Z^n = \bigcup_{j\in[m]} Z_j^n.$$

Algorithm 1: Multi-Party Data Representation.

Input: For each input party P_i ($i \in \{1, \ldots, n-2\}$): paired dataset $S_i = \{(x_j^i, u_j^i)\}_{j\in[m]}$ with $X_i = \{x_j^i\}_{j\in[m]}$ and $U_i = \{u_j^i\}_{j\in[m]}$.
Output: OKVS encodings $\{(D_i, \widetilde{D}_i)\}_{i=1}^{n-2}$.

Participant P_1
Jointly establish PRF keys $\{k_i\}_{i=2}^{n-2}$ with parties $\{P_i\}_{i=2}^{n-2}$ using Section V-B;
for $(x_j^1, u_j^1) \in S_1$ **do**
  $\alpha_j^1 \leftarrow \bigoplus_{i=2}^{n-2} \mathrm{PRF}_{k_i}(x_j^1)$;
  $\beta_j^1 \leftarrow \bigoplus_{i=2}^{n-2} \mathrm{PRF}_{k_i}(u_j^1)$;
$D_1 \leftarrow \mathsf{OKVS.Encode}(\{(x_j^1, \alpha_j^1)\}_{j\in[m]})$;
$\widetilde{D}_1 \leftarrow \mathsf{OKVS.Encode}(\{(u_j^1, \beta_j^1)\}_{j\in[m]})$;
Send $(D_1, \widetilde{D}_1)$ to P_n;

Participant P_i, $2 \le i \le n-2$
Use the jointly established PRF key k_i shared with P_1;
for $(x_j^i, u_j^i) \in S_i$ **do**
  compute $\mathrm{PRF}_{k_i}(x_j^i)$;
  compute $\mathrm{PRF}_{k_i}(u_j^i)$;
$D_i \leftarrow \mathsf{OKVS.Encode}(\{(x_j^i, \mathrm{PRF}_{k_i}(x_j^i))\}_{j\in[m]})$;
$\widetilde{D}_i \leftarrow \mathsf{OKVS.Encode}(\{(u_j^i, \mathrm{PRF}_{k_i}(u_j^i))\}_{j\in[m]})$;
Send $(D_i, \widetilde{D}_i)$ to P_{n-1};
return $\{(D_i, \widetilde{D}_i)\}_{i=1}^{n-2}$;
// P_{n-1} receives $\{(D_i, \widetilde{D}_i)\}_{i=2}^{n-2}$, and P_n receives $(D_1, \widetilde{D}_1)$.

Algorithm 2: Bicentric Mislabeled Resolution.

Input: P_{n-1} holds $\{(D_i, \widetilde{D}_i)\}_{i=2}^{n-2}$ and S_{n-1}; P_n holds $(D_1, \widetilde{D}_1)$ and S_n.
Output: Mislabeled-record set $\mathcal{C}$.

Bicentric party P_{n-1};
Initialize an empty token-to-record map π_{n-1};
for $(x_j^{n-1}, u_j^{n-1}) \in S_{n-1}$ **do**
  $b_j^{n-1} \leftarrow \bigoplus_{i=2}^{n-2} \mathsf{OKVS.Decode}(D_i, x_j^{n-1})$;
  $\widetilde{b}_j^{n-1} \leftarrow \bigoplus_{i=2}^{n-2} \mathsf{OKVS.Decode}(\widetilde{D}_i, u_j^{n-1})$;
  $Z_j^{n-1} \leftarrow \{\, b_j^{n-1} \parallel \mathsf{Prefix}_\ell(\widetilde{b}_j^{n-1}) \,\}_{\ell=1}^{\lambda_u}$;
  Store $\pi_{n-1}(z) = x_j^{n-1}$ for each $z \in Z_j^{n-1}$;
$Z^{n-1} \leftarrow \bigcup_{j\in[m]} Z_j^{n-1}$;

Bicentric party P_n;
Initialize an empty token-to-record map π_n;
for $(x_j^n, u_j^n) \in S_n$ **do**
  $b_j^n \leftarrow \mathsf{OKVS.Decode}(D_1, x_j^n)$;
  $\widetilde{b}_j^n \leftarrow \mathsf{OKVS.Decode}(\widetilde{D}_1, u_j^n)$;
  $Z_j^n \leftarrow \{\, b_j^n \parallel \mathsf{FlipLast}(\mathsf{Prefix}_\ell(\widetilde{b}_j^n)) \,\}_{\ell=1}^{\lambda_u}$;
  Store $\pi_n(z) = x_j^n$ for each $z \in Z_j^n$;
$Z^n \leftarrow \bigcup_{j\in[m]} Z_j^n$;

Set matching and reporting;
$M \leftarrow Z^{n-1} \cap Z^n$;
$\mathcal{C} \leftarrow \{\pi_n(z) : z \in M\}$ according to the output policy;
return $\mathcal{C}$;

The matching step computes $M = Z^{n-1} \cap Z^n$,and maps the matched tokens in M to the corresponding mislabeled records according to the prescribed output policy. The token-set sizes are determined by public parameters and captured by $\mathsf{size}_{\mathsf{msg}}$, while the matched-token cardinality before duplicate removal and the match/no-match outcomes visible to the bicentric parties are captured by μ in $\mathcal{L}_{\mathsf{Bic\text{-}DC}}$. Public abort behavior is separately captured by abort.

We next characterize why a matched token corresponds to a mislabeled record. For any $j, j' \in [m]$, each element of Z_j^{n-1} and $Z_{j'}^n$ is constructed by concatenating a masked record encoding with a complete or bit-flipped prefix of the corresponding masked label encoding. Therefore, an intersection arises if and only if two conditions hold simultaneously: (i) the record encodings are identical, i.e., $b_j^{n-1} = b_{j'}^n$; and (ii) the label encodings differ, i.e., $\widetilde{b}_j^{n-1} \neq \widetilde{b}_{j'}^n$. By the prefix-flipping property, this condition can be summarized as

$$Z_{j'}^n \cap Z_j^{n-1} \neq \varnothing \iff \left(b_j^{n-1} = b_{j'}^n\right) \wedge \left(\widetilde{b}_j^{n-1} \neq \widetilde{b}_{j'}^n\right).$$

Thus, the global matched-token set $M = Z^{n-1} \cap Z^n$ identifies records that are common to the bicentric parties and have inconsistent labels.

Through this procedure, Bic-DC detects mislabeled records without explicitly revealing the full record intersection. Consistently labeled overlaps do not produce flipped-prefix matches, and non-overlapping records do not share the same masked record encoding across the bicentric parties. Therefore, the protocol reveals no information beyond the prescribed mislabeled-record output and the explicitly modeled leakage in $\mathcal{L}_{\mathsf{Bic\text{-}DC}}$.

E. Illustrative Example

To provide a concrete understanding of Bic-DC, we present a toy example with five participants, each holding three records and their labels, as shown in Fig. 5. The example covers three representative cases: (i) an all-party common record with a consistent label, e.g., $a : 1$; (ii) an all-party common record with conflicting labels, e.g., record b; and (iii) non-overlapping records. Bic-DC reveals only the inconsistent overlap, namely the red-highlighted record b in Fig. 5, while keeping consistent overlaps and non-overlapping records hidden except for the explicitly modeled leakage.

a) Step 1: Multi-party OKVS encoding: Each non-bicentric participant locally evaluates PRFs on its records and labels. In this five-party example, P_1 uses the pairwise PRF keys k_2 and k_3, jointly established with P_2 and P_3, respectively. It evaluates both PRFs on each local record or label and XORs the outputs, e.g., $F_{k_2}(x) \oplus F_{k_3}(x)$, so that identical inputs can be reconstructed in a common pseudorandom space by the bicentric parties, while distinct inputs remain unlinkable. Meanwhile, P_2 and P_3 compute $F_{k_2}(\cdot)$ and $F_{k_3}(\cdot)$ on their local inputs and encode the resulting key–value pairs via OKVS. Finally, P_1 produces $(D_1, \widetilde{D}_1)$ and sends them to P_5, whereas P_2 and P_3 generate $(D_2, \widetilde{D}_2)$ and $(D_3, \widetilde{D}_3)$ and send them to P_4. The bicentric

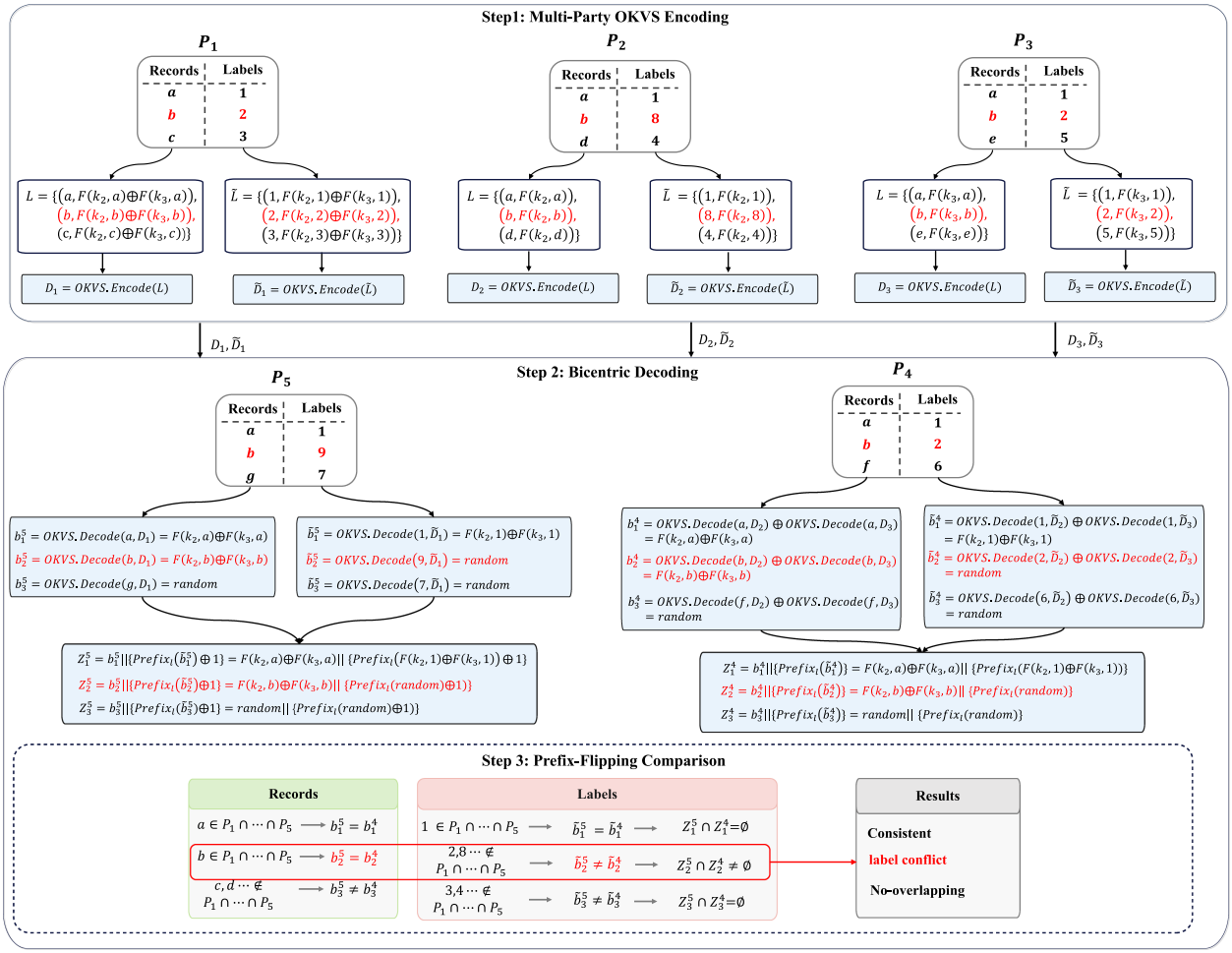


Fig. 5. Illustrative example of Bic-DC, where only the mislabeled record b is revealed, while consistent and disjoint records remain concealed.

parties, P_4 and P_5, only receive OKVS encodings in this phase and do not submit encodings themselves.

b) Step 2: Bicentric decoding: After receiving the OKVS encodings, P_4 and P_5 decode masked representations of their local records and labels, producing prefix-token sets $Z_j^4 = \{b_j^4 \parallel \mathsf{Prefix}_\ell(\widetilde{b}_j^4)\}_{\ell\in[\lambda_u]}$ and $Z_j^5 = \{b_j^5 \parallel \mathsf{FlipLast}(\mathsf{Prefix}_\ell(\widetilde{b}_j^5))\}_{\ell\in[\lambda_u]}$. Records that appear in all parties yield matching record tags, while label tags coincide only when the corresponding labels are consistent. For example, the consistently labeled record a has $b_1^4 = b_1^5$ and $\widetilde{b}_1^4 = \widetilde{b}_1^5$, while the conflicting record b has $b_2^4 = b_2^5$ but $\widetilde{b}_2^4 \neq \widetilde{b}_2^5$. Thus, b is matched at the record level but inconsistent at the label level, and will be detected in Step 3.

c) Step 3: Prefix-flipping comparison: Using the decoded prefix-token sets, the bicentric parties perform the prefix-flipping comparison described in Section III-B. A match occurs only when the record tags match and the label tags differ, i.e., $b_j^4 = b_{j'}^5$ and $\widetilde{b}_j^4 \neq \widetilde{b}_{j'}^5$. Consistently labeled records produce no match because their label tags are equal, and non-overlapping records cannot match because their record tags differ. In the example, $Z_2^4 \cap Z_2^5 \neq \emptyset$ for record b, so it is reported as mislabeled, whereas record a and non-overlapping records do not yield intersections. Thus, Bic-DC reports only label-conflicting all-party overlaps while excluding consistent and non-overlapping records.

In this example, record b satisfies $b_2^4 = b_2^5$ but $\widetilde{b}_2^4 \neq \widetilde{b}_2^5$, so $Z_2^4 \cap Z_2^5 \neq \emptyset$ and b is reported as mislabeled; by contrast, the consistent record a and non-overlapping records such as f and g do not produce matched prefix tokens. Thus, only the label-conflicting all-party overlap is revealed.

F. Complexity Analysis

We analyze the computational and communication complexity of the Bic-DC protocol with n parties P_i, each holding m record-label pairs.

a) Computational Complexity: The computation consists of four stages: decentralized PRF-key setup, PRF evaluation and OKVS encoding by the input parties, OKVS decoding by the bicentric parties, and prefix-token generation followed by set matching for mislabeled detection.

- *Decentralized PRF-key setup:* For each non-bicentric party P_i with $2 \le i \le n-2$, P_1 and P_i jointly derive one PRF key using two commitments, two openings, and one hash-based key derivation. The total setup cost is $O(n)$

lightweight cryptographic operations. Since this is a one-time cost independent of the dataset size m, it does not affect the asymptotic online computation of Bic-DC.

- P_1. For each pair (x_j^1, u_j^1), P_1 evaluates the $n-3$ PRFs on both the record and the label, resulting in $O((n-3)m)$ PRF invocations. It then encodes two OKVS structures over m items, each costing $O(m)$. The overall online computation of P_1 is therefore $O(nm)$.
- P_i $(2 \le i \le n-2)$. Each P_i evaluates its PRF on its m records and m labels and constructs two OKVS instances of size m. The per-party online computation is $O(m)$.
- Bicentric party P_{n-1}. P_{n-1} decodes all $(n-3)$ OKVS structures received from parties P_i $(2 \le i \le n-2)$, each over m local elements, resulting in $O((n-3)m)$ decoding operations. For each decoded label $\widetilde{b}_j^{n-1}$, P_{n-1} generates λ_u prefix tokens, which incurs $O(m\lambda_u)$ additional work. Thus, the total computation at P_{n-1} is $O(nm + m\lambda_u)$.
- Bicentric party P_n. P_n decodes only the OKVS structures sent by P_1, with cost $O(m)$. It similarly expands each decoded label $\widetilde{b}_j^n$ into λ_u bit-flipped prefix tokens, contributing $O(m\lambda_u)$ work. Hence, the total computation at P_n is $O(m + m\lambda_u)$.
- *Set matching:* The two bicentric parties perform set matching over prefix-token sets of size $O(m\lambda_u)$. Using hashing-based set intersection, this step costs $O(m\lambda_u)$ expected time; using a sorting-based implementation, it costs $O(m\lambda_u \log(m\lambda_u))$. In our analysis, we use the standard hashing-based implementation and count this step as $O(m\lambda_u)$ expected time.

b) Communication Complexity: The communication overhead arises from three sources: (i) decentralized PRF-key setup, (ii) transmitting OKVS encodings from the input parties to the bicentric parties, and (iii) set matching between the bicentric parties.

- *Decentralized PRF-key setup:* For each P_i with $2 \le i \le n-2$, the setup requires two commitments and two openings between P_1 and P_i. Assuming each commitment and opening has size $O(\kappa)$, the total setup communication is $O(n\kappa)$ bits. This cost is incurred only once per cleaning session and is independent of m.
- Input parties (P_1 and P_i, $2 \le i \le n-2$). Each input party transmits two OKVS structures $(D_i, \widetilde{D}_i)$ of size $O(m)$ to its designated bicentric party, namely $P_1 \to P_n$ and $P_i \to P_{n-1}$. Hence, the per-party communication is $O(m)$, and across all $(n-2)$ input parties the total communication is $O((n-2)m)$.
- Bicentric parties. P_{n-1} receives $O((n-3)m)$ OKVS encodings from parties P_i $(2 \le i \le n-2)$, while P_n receives $O(m)$ encodings from P_1. During the final set-matching phase, the bicentric parties compare prefix-token sets of size $O(m\lambda_u)$. Therefore, the communication for bicentric matching is $O(m\lambda_u)$, depending on the concrete set-matching implementation. Overall, the bicentric-side communication is $O((n-3)m + m\lambda_u)$ for P_{n-1} and $O(m + m\lambda_u)$ for P_n.

VI. Security Analysis

We analyze the security of Bic-DC in the semi-honest model under the non-collusion assumption stated in Section IV-B. The proof relies on the pseudorandomness of the PRF, the obliviousness of OKVS, and the correctness of the prefix-flipping based bicentric matching procedure.

Theorem 1 (Security of Bic-DC): Assuming a secure PRF and a correct and oblivious OKVS construction, Bic-DC securely realizes the leakage-aware ideal functionality $\mathcal{F}_{\mathsf{MCDC}}$ with leakage profile $\mathcal{L}_{\mathsf{Bic\text{-}DC}}$ in the semi-honest model under the stated non-collusion assumption.

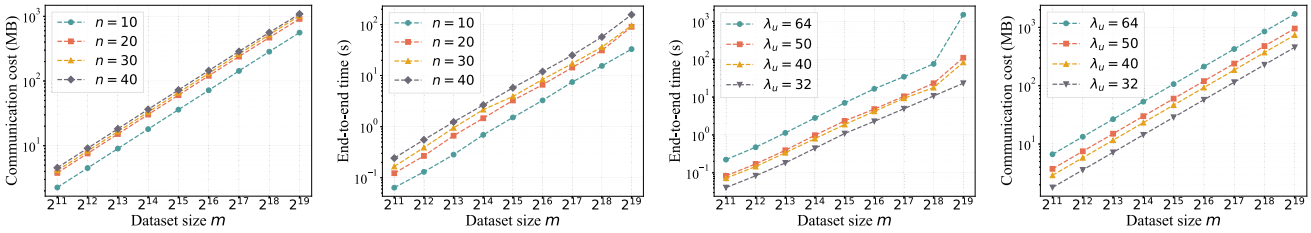
More precisely, for every allowed static semi-honest corruption pattern, there exists a probabilistic polynomial-time simulator that generates a view computationally indistinguishable from the real execution using only the corrupted parties' inputs, the prescribed output C, and $\mathcal{L}_{\mathsf{Bic\text{-}DC}}$.

Proof sketch: We show that the view of each allowed corrupted party can be simulated from its input, the prescribed output C, and the explicit leakage profile. The leakage profile includes public parameters, fixed message sizes, the bicentric matching transcript visible during resolution, and public abort behavior. Thus, message-length leakage, match/no-match outcomes over PRF-masked tokens, matched-token cardinality, and abort events are explicitly modeled rather than implicitly hidden by OKVS obliviousness.

Non-bicentric parties: A non-bicentric party only performs local PRF evaluation, OKVS encoding, and one-time submission of its encodings to the bicentric parties. It receives no data-dependent messages after submission. Hence, a simulator can reproduce its view using its local input, randomness, and public parameters. Since messages from other parties are not visible to it, the simulated view is identically distributed to the real view.

Bicentric parties: A bicentric party's view consists of its local dataset, the OKVS encodings received from non-bicentric parties, locally decoded masked values, and the bicentric matching transcript. For honest parties' encodings, the simulator samples OKVS objects with the same public sizes and random values. By OKVS obliviousness, these simulated encodings are computationally indistinguishable from real encodings whose values are PRF outputs or XORs of PRF outputs. For tokens corresponding to the prescribed mislabeled-record output, the simulator programs the simulated masked values consistently with the output C and the leakage term μ. All remaining decoded values are sampled as independent pseudorandom strings of the appropriate length, which is indistinguishable from the real execution by PRF security and OKVS obliviousness.

The simulator uses μ in $\mathcal{L}_{\mathsf{Bic\text{-}DC}}$ to reproduce the bicentric matching transcript, including the number of matched PRF-masked tokens and the match/no-match outcomes visible to the bicentric parties. It also uses `abort` to simulate normal termination or public abort behavior. Since all transmitted encodings have fixed sizes determined by public parameters, the simulated communication pattern is consistent with the real execution. Therefore, the simulated view of any corrupted bicentric party is computationally indistinguishable from its real view.



(a) Comm. vs. under varying n (b) Runtime vs. under varying n (c) Runtime vs. under varying λ_u (d) Comm. vs. under varying λ_u

Fig. 6. System-level performance of Bic-DC under varying dataset sizes. Subfigures (a) and (b) fix the label-encoding length to $\lambda_u = 32$ and vary the number of participants n. Subfigures (c) and (d) fix the number of participants to $n = 4$ and vary the label-encoding length λ_u. The results report end-to-end online runtime and total communication cost.

TABLE II
OKVS MEMORY FOOTPRINT AND ENCODING/DECODING CONSTANTS

m	Table size (MB)	Encode (s/10^6 items)	Decode (s/10^6 items)
2^{12}	0.083	0.105	0.358
2^{14}	0.317	0.213	0.262
2^{16}	1.24	0.254	0.259
2^{18}	4.93	0.374	0.283
2^{20}	19.69	0.725	0.212

Correctness: For each record or label value e, the coordinator-side encoding and the corresponding non-bicentric encodings are derived from the same PRF outputs. Thus, if the same value e is present in all required parties, both bicentric parties reconstruct the same masked encoding; otherwise, the reconstructed values are computationally indistinguishable from random. The bicentric parties then concatenate record encodings with complete or bit-flipped prefixes of label encodings. By the prefix-flipping property, under the same masked record encoding, a token match occurs if and only if the corresponding label encodings differ. Therefore, except with negligible probability from PRF collisions or OKVS failure, Bic-DC outputs exactly the mislabeled-record set C.

Output privacy: Consistently labeled overlaps do not produce flipped-prefix matches, and non-overlapping records do not yield matching record encodings across the bicentric parties. Thus, they are excluded from the output. By PRF security and OKVS obliviousness, their encodings are computationally indistinguishable from random values to any allowed corrupted party, except for the explicitly stated leakage $\mathcal{L}_{\text{Bic-DC}}$. □

VII. PERFORMANCE EVALUATION

We evaluate Bic-DC from three perspectives. First, we characterize its system-level performance, including correctness, end-to-end runtime, communication cost, OKVS memory footprint, and scalability under varying dataset sizes, label-encoding lengths, and participant numbers. Second, we compare Bic-DC with two-party privacy-preserving data-cleaning baselines to quantify the cost reduction achieved by bicentric resolution over repeated pairwise executions. Third, we compare Bic-DC with outsourced/cloud-assisted MPSI protocols and recent efficient MPSI constructions. Since MPSI protocols only compute all-party intersections and do not support label-conflict detection, this comparison measures the additional cost incurred by Bic-DC to provide task-specific inconsistency detection while avoiding full-intersection disclosure.

A. Experimental Setup

Implementation and environment: All protocols are implemented in C++ with multi-threading enabled and compiled using `g++` with `-O3` optimization. Unless otherwise stated, experiments are conducted on a dedicated benchmarking server equipped with two Intel Xeon Gold 6348 CPUs (56 physical cores, 112 threads, 2.60 GHz) and 256 GB RAM, running Ubuntu 22.04.3 LTS with Linux kernel 5.15. Network conditions are controlled using the Linux `tc/netem` module. We evaluate three representative settings: LAN (1 Gbps, 0.2 ms RTT), moderate WAN (100 Mbps, 20 ms RTT), and constrained WAN (10 Mbps, 20 ms RTT).

Baselines: For two-party privacy-preserving data-cleaning baselines, we implement two representative OPRF-based constructions following the framework of Blass and Kerschbaum [8]. The first is a KKRT-based scheme, denoted KKRT-PDC$_2$, which integrates the KKRT-OPRF implementation from libOTe [47]. The second is a VOLE-based scheme, denoted VOLE-PDC$_2$, which uses the VOLE-OPRF implementation from volePSI [45]. KKRT-PDC$_2$ is optimized for computational efficiency, whereas VOLE-PDC$_2$ is designed to reduce communication overhead. Together, they represent natural pairwise extensions of privacy-preserving collaborative data cleaning.

For multi-party comparison, we consider a state-of-the-art MPSI protocol [35] and MPSA [13], a representative outsourced/cloud-assisted MPSI protocol. Since these protocols compute only all-party intersections and do not support label-conflict detection, the comparison measures the additional cost incurred by Bic-DC for realizing task-specific mislabeled-record detection while avoiding full-intersection disclosure.

Libraries and reproducibility: Our implementation builds on libOTe [47] and OKVS constructions following Raghuraman et al. [45]. All source code, scripts, and reproduction instructions are publicly available at: https://github.com/savannahaa/BIR-BDC.

TABLE III
COMPARISON WITH PAIRWISE DATA-CLEANING EXTENSIONS UNDER VARYING DATASET SIZES m AND LABEL-ENCODING LENGTHS λ_u. KKRT-PDC$_2$ AND VOLE-PDC$_2$ ARE TWO-PARTY BASELINES, WHILE BIC-DC IS EVALUATED WITH $n = 4, 6, 8$. BOLD VALUES INDICATE THE BEST RESULT FOR EACH METRIC.

m	λ_u	KKRT-PDC$_2$ ($n=2$)				VOLE-PDC$_2$ ($n=2$)				Bic-DC ($n=4$)				Bic-DC ($n=6$)				Bic-DC ($n=8$)			
		Runtime			Comm.	Runtime			Comm.	Runtime			Comm.	Runtime			Comm.	Runtime			Comm.
		1 Gbps	100 Mbps	10 Mbps	(MB)	1 Gbps	100 Mbps	10 Mbps	(MB)	1 Gbps	100 Mbps	10 Mbps	(MB)	1 Gbps	100 Mbps	10 Mbps	(MB)	1 Gbps	100 Mbps	10 Mbps	(MB)
2^{11}	32	0.32	0.77	5.77	9.47	0.35	1.04	7.85	6.13	**0.06**	**0.19**	**1.55**	**1.79**	0.08	0.22	1.69	1.95	0.08	0.24	1.83	2.10
	40	0.38	1.09	8.15	12.06	0.60	1.47	10.16	9.80	**0.09**	**0.31**	**2.50**	**2.90**	0.11	0.34	2.65	3.06	0.12	0.36	2.78	3.21
	50	0.47	1.35	10.17	14.75	0.71	1.77	12.39	12.26	**0.11**	**0.40**	**3.23**	**3.75**	0.13	0.43	3.37	3.90	0.15	0.45	3.51	4.05
	64	0.92	2.31	16.21	22.52	1.02	2.64	18.85	19.30	0.49	0.99	**6.02**	**6.65**	**0.46**	**0.97**	6.11	6.81	0.63	1.14	6.22	6.73
2^{12}	32	0.60	1.49	10.31	17.69	0.81	2.08	14.82	12.26	**0.12**	**0.39**	**3.10**	**3.59**	0.15	0.44	3.38	3.90	0.17	0.49	3.66	4.20
	40	0.73	2.14	16.26	22.84	1.04	2.68	19.13	19.61	**0.19**	**0.62**	**5.01**	**5.81**	0.22	0.68	5.30	6.11	0.25	0.73	5.58	6.42
	50	0.90	2.67	20.31	28.21	1.24	3.27	23.58	24.51	**0.24**	**0.80**	**6.46**	**7.50**	0.27	0.86	6.75	7.80	0.30	0.91	7.04	8.11
	64	1.81	4.59	32.39	43.73	1.89	5.04	36.53	38.60	1.39	2.40	**12.42**	**13.31**	**1.25**	**2.28**	12.56	13.62	1.70	2.72	12.88	14.26
2^{13}	32	1.20	2.97	20.62	34.11	1.08	3.54	28.10	24.51	**0.23**	**0.78**	**6.19**	**7.18**	0.31	0.90	6.78	7.79	0.36	1.00	7.34	8.40
	40	1.53	4.28	32.51	44.39	1.93	5.13	37.09	39.22	**0.40**	**1.28**	**10.05**	**11.61**	0.49	1.41	10.64	12.23	0.56	1.53	11.22	12.84
	50	1.80	5.33	40.62	55.15	2.37	6.34	46.05	49.02	**0.56**	**1.69**	**13.01**	**14.99**	0.60	1.78	13.56	15.60	0.66	1.89	14.13	16.21
	64	**2.92**	8.48	64.07	86.11	3.90	10.10	72.10	77.21	2.93	**4.94**	**25.03**	**26.61**	3.11	5.16	25.72	27.27	3.25	5.29	25.22	26.92
2^{14}	32	2.31	5.84	41.14	65.87	2.09	6.84	54.26	49.02	**0.54**	**1.63**	**12.46**	**14.35**	0.74	1.91	13.67	15.57	0.84	2.11	14.79	16.80
	40	2.91	8.56	65.03	87.43	3.43	9.73	72.68	78.43	**0.94**	**2.70**	**20.23**	**23.23**	1.08	2.92	21.38	24.45	1.48	3.42	22.80	25.68
	50	3.61	10.67	81.26	108.94	4.22	12.06	90.50	98.04	**1.27**	**3.54**	**26.17**	**29.98**	1.38	3.73	27.29	31.20	1.76	4.20	28.68	32.46
	64	6.92	18.04	129.22	170.97	8.24	20.55	143.65	154.42	**5.45**	**9.47**	**49.66**	**53.23**	6.36	10.47	51.58	54.45	6.45	10.52	51.17	53.85
2^{15}	32	4.56	11.71	82.31	130.42	4.22	13.61	107.51	98.04	**1.21**	**3.38**	**25.04**	**28.69**	1.60	3.95	27.46	31.13	1.88	4.41	29.76	33.57
	40	5.82	17.11	130.06	173.65	8.35	20.85	145.88	156.87	**2.13**	**5.64**	**40.70**	**46.44**	2.47	6.16	43.07	48.88	2.90	6.77	45.52	51.32
	50	7.25	21.37	162.55	216.70	10.57	26.17	182.20	196.09	**2.80**	**7.33**	**52.58**	**59.94**	2.97	7.68	54.77	62.38	3.53	8.43	57.36	64.82
	64	14.43	36.67	259.03	340.62	14.56	39.08	284.32	308.83	**11.69**	**19.73**	**100.09**	**106.44**	12.56	20.78	102.98	108.89	11.99	20.12	101.43	107.69
2^{16}	32	9.36	23.48	164.66	259.61	8.97	27.67	214.59	196.08	**2.76**	**7.09**	**50.38**	**57.34**	3.55	8.24	55.19	62.19	4.13	9.19	59.80	67.03
	40	11.66	34.24	260.13	345.98	13.29	38.20	287.31	313.74	**4.56**	**11.57**	**81.66**	**92.84**	5.50	12.88	86.63	97.69	6.18	13.92	91.33	102.53
	50	14.77	42.71	325.07	432.12	18.42	49.54	360.07	392.14	**5.93**	**14.98**	**105.46**	**119.84**	6.45	15.85	110.00	124.69	7.41	17.19	114.98	129.53
	64	27.76	72.28	516.95	679.97	41.99	90.94	580.52	617.66	23.33	39.20	**199.89**	**212.84**	28.43	44.65	209.21	217.69	**21.38**	**37.64**	200.24	215.38
2^{17}	32	15.33	43.57	325.93	517.99	25.93	63.23	436.18	392.17	**5.88**	**14.52**	**101.00**	**114.55**	7.77	17.14	110.83	124.09	8.73	18.81	119.71	133.64
	40	24.09	69.26	521.04	690.75	29.22	78.95	576.29	627.47	**9.53**	**23.54**	**163.62**	**185.54**	11.42	26.15	173.44	195.09	13.13	28.58	183.08	204.64
	50	29.35	85.82	650.55	863.00	39.15	101.27	722.63	784.34	**12.91**	**30.93**	**211.84**	**239.54**	13.76	32.57	220.62	249.09	15.59	35.11	230.38	258.60
	64	57.01	145.95	1035.39	1358.78	94.73	192.56	1170.88	1235.33	**43.25**	**75.37**	**396.65**	**425.54**	54.23	87.08	415.56	435.08	46.27	78.79	404.00	430.76
2^{18}	32	38.27	94.74	659.46	1034.81	50.85	125.35	870.42	784.33	**13.06**	**30.31**	**202.85**	**228.54**	16.22	34.87	221.42	247.09	18.95	39.00	239.55	265.63
	40	47.21	137.56	1041.16	1380.32	73.76	173.15	1166.98	1254.94	**20.52**	**48.50**	**328.25**	**370.54**	23.70	53.08	346.83	389.09	26.78	57.56	365.31	407.63
	50	60.44	173.38	1302.83	1724.92	93.97	218.17	1460.11	1568.67	**26.39**	**62.52**	**423.81**	**478.54**	30.34	67.87	443.16	497.09	31.82	70.76	460.04	515.63
	64	119.34	297.26	2076.13	2716.57	214.84	410.44	2366.37	2470.66	**83.52**	**147.73**	**789.87**	**850.53**	108.77	174.39	830.52	869.08	110.11	175.16	825.58	861.52
2^{19}	32	68.07	181.02	1310.46	2068.65	148.56	297.51	1786.93	1568.67	**26.86**	**61.23**	**404.87**	**455.17**	33.71	70.73	440.93	490.34	39.86	79.53	476.29	525.52
	40	94.98	275.69	2082.80	2759.77	186.06	384.76	2371.79	2509.87	**42.82**	**98.63**	**656.68**	**739.17**	57.49	115.95	700.56	774.34	56.09	117.21	728.37	809.52
	50	119.29	345.18	2604.05	3449.10	287.05	535.38	3018.68	3137.34	**105.83**	**177.95**	**899.08**	**955.17**	140.80	215.57	963.25	990.34	131.06	208.49	982.73	1025.52
	64	233.76	589.53	4147.27	5432.64	662.63	1053.78	4965.28	4941.31	**175.65**	**303.95**	**1586.76**	**1699.15**	266.14	397.08	1706.43	1734.30	208.84	338.93	1639.70	1723.04

B. System-Level Performance of Bic-DC

Impact of participant number: Fig. 6(a) and (b) report the communication cost and end-to-end runtime of Bic-DC under different participant numbers, with the label-encoding length fixed to $\lambda_u = 32$. As the dataset size m increases from 2^{11} to 2^{19}, both communication and runtime grow approximately linearly, which is consistent with the $O(nm + m\lambda_u)$ complexity. For a fixed m, increasing n leads to higher communication and runtime because the bicentric party P_{n-1} receives and decodes more OKVS tables from non-bicentric parties. Nevertheless, the growth remains controlled and does not exhibit quadratic behavior, confirming that Bic-DC avoids repeated pairwise executions among all participants.

Impact of label-encoding length: Fig. 6(c) and (d) show the effect of varying the label-encoding length λ_u when the number of participants is fixed to $n = 4$. Larger λ_u increases the number of prefix tokens generated for each decoded label, thereby increasing both prefix-token matching time and communication. The communication curves grow steadily with m and become larger for longer label encodings, matching the expected $O(m\lambda_u)$ prefix-token expansion cost. The runtime curves show the same trend: shorter encodings such as $\lambda_u = 32$ incur the lowest overhead, while longer encodings introduce higher cost due to additional prefix generation and matching operations.

Correctness under truncated encodings: The prefix-flipping comparison is deterministic given collision-free masked encodings; hence, detection errors mainly stem from accidental collisions caused by truncating PRF outputs. By the standard birthday bound, for Q pseudorandom tokens, the collision probability is at most about $Q^2/2^{\lambda_u+1}$, which becomes negligible for $\lambda_u = 128$. Empirically, 24-bit encodings achieve about 96.9% exact success, 32-bit encodings already reach 99.987%, and $\lambda_u \geq 40$ yields essentially perfect detection in our tests. Therefore, we use $\lambda_u = 128$ in the main experiments to ensure negligible collision-induced errors.

OKVS microbenchmark: Table II reports the concrete memory footprint and encoding/decoding constants of the OKVS implementation used in Bic-DC. The OKVS table size remains moderate across all tested settings and reaches 19.69 MB at $m = 2^{20}$. The measured encoding and decoding costs are also small in concrete terms: at the largest tested size, encoding one OKVS table costs 0.725 s per 10^6 items, while decoding costs 0.212 s per 10^6 queried keys. In the largest scalability setting with $n = 110$ and $m = 2^{20}$, P_n stores two OKVS tables, while P_{n-1} stores $2(n-3) = 214$ tables, corresponding to 39.38 MB and about 4.11 GB of OKVS storage, respectively. These results show that the OKVS layer has practical concrete constants and that its memory footprint remains manageable in our largest experiments.

TABLE IV
SYSTEM-LEVEL COMPARISON BETWEEN THE MPSI CORE OF BIC-DC, STATE-OF-THE-ART MPSI, AND OUTSOURCED/CLOUD-ASSISTED MPSA UNDER DIFFERENT DATA SIZES, PARTICIPANT NUMBERS, AND BANDWIDTH SETTINGS. HIGHLIGHTED CELLS INDICATE THE BEST RESULT FOR EACH METRIC.

m	n	Bic-DC				MPSI				MPSA			
		1 Gbps(s)	100 Mbps(s)	10 Mbps(s)	Comm.(MB)	1 Gbps(s)	100 Mbps(s)	10 Mbps(s)	Comm.(MB)	1 Gbps(s)	100 Mbps(s)	10 Mbps(s)	Comm.(MB)
2^{11}	4	0.02	0.02	0.13	0.14	0.01	0.02	0.16	0.19	0.11	0.12	0.23	0.14
	6	0.02	0.03	0.20	0.22	0.01	0.03	0.24	0.28	0.14	0.16	0.32	0.22
	8	0.02	0.04	0.27	0.29	0.01	0.04	0.31	0.36	0.20	0.22	0.44	0.29
	20	0.06	0.11	0.69	0.76	0.03	0.09	0.75	0.87	0.52	0.58	1.16	0.77
2^{12}	4	0.02	0.04	0.25	0.28	0.01	0.04	0.33	0.38	0.20	0.22	0.44	0.28
	6	0.03	0.06	0.39	0.43	0.02	0.06	0.47	0.55	0.22	0.26	0.58	0.43
	8	0.04	0.08	0.53	0.59	0.02	0.07	0.62	0.72	0.37	0.42	0.85	0.57
	20	0.11	0.23	1.37	1.51	0.06	0.19	1.50	1.73	0.98	1.09	2.20	1.48
2^{13}	4	0.03	0.07	0.50	0.56	0.02	0.08	0.65	0.76	0.41	0.45	0.88	0.57
	6	0.05	0.12	0.77	0.87	0.03	0.12	0.94	1.10	0.33	0.39	1.04	0.85
	8	0.09	0.18	1.07	1.17	0.05	0.15	1.24	1.43	0.50	0.58	1.44	1.14
	20	0.24	0.47	2.75	3.02	0.11	0.37	2.99	3.46	1.88	2.09	4.27	2.88
2^{14}	4	0.06	0.15	0.99	1.12	0.05	0.17	1.31	1.51	0.48	0.57	1.41	1.13
	6	0.11	0.25	1.55	1.73	0.09	0.26	1.91	2.19	1.16	1.29	2.57	1.69
	8	0.18	0.36	2.13	2.35	0.11	0.33	2.49	2.86	0.84	1.01	2.71	2.26
	20	0.73	1.19	5.74	6.04	0.25	0.77	5.99	6.92	3.01	3.44	7.74	5.70
2^{15}	4	0.12	0.29	1.98	2.23	0.11	0.34	2.63	3.03	1.03	1.20	2.90	2.25
	6	0.23	0.49	3.11	3.46	0.21	0.54	3.85	4.38	2.31	2.52	5.11	3.38
	8	0.39	0.75	4.29	4.69	0.24	0.67	5.00	5.73	1.66	2.00	5.40	4.51
	20	1.52	2.43	11.55	12.07	0.65	1.70	12.14	13.83	6.97	7.82	16.37	11.32
2^{16}	4	0.26	0.60	3.97	4.46	0.27	0.72	5.29	6.05	2.03	2.37	5.77	4.50
	6	0.51	1.03	6.25	6.92	0.38	1.04	7.65	8.75	4.26	4.77	9.87	6.76
	8	0.80	1.50	8.59	9.38	0.53	1.40	10.05	11.45	5.74	6.42	13.23	9.01
	20	2.63	4.45	22.67	24.14	0.94	3.03	23.91	27.65	14.74	16.44	33.48	22.57
2^{17}	4	0.58	1.26	7.91	8.92	0.57	1.47	10.61	12.10	6.40	7.08	13.88	9.00
	6	1.26	2.30	12.75	13.84	1.08	2.46	15.61	17.50	9.21	10.24	20.44	13.51
	8	2.37	3.79	17.95	18.76	1.25	2.98	20.27	22.90	12.01	13.38	26.97	18.01
	20	6.01	9.65	46.10	48.28	2.29	6.46	48.22	55.31	29.13	32.53	66.56	45.07
2^{18}	4	1.26	2.61	16.08	17.84	1.63	3.46	21.73	24.20	12.75	14.11	27.70	18.00
	6	2.51	4.60	25.49	27.68	2.48	5.12	31.55	35.00	13.85	15.89	36.28	27.01
	8	3.95	6.78	35.10	37.52	3.16	6.62	41.20	45.80	23.95	26.67	53.86	36.01
	20	12.26	19.55	92.48	96.56	5.23	13.58	97.08	110.61	57.78	64.58	132.58	90.07
2^{19}	4	2.90	5.60	32.53	35.68	3.74	7.39	43.93	48.40	21.57	24.29	51.47	36.00
	6	5.61	9.79	51.19	55.36	4.89	10.17	63.02	70.00	22.76	26.84	67.61	54.01
	8	8.26	13.92	70.58	75.04	6.42	13.34	82.49	91.60	32.27	37.71	92.07	72.01
	20	25.85	40.43	186.23	193.12	15.95	32.65	199.65	221.21	95.08	108.68	244.63	180.07

C. Comparison With Pairwise Data-Cleaning Extensions

Table III compares Bic-DC with KKRT-PDC$_2$ and VOLE-PDC$_2$ under varying dataset sizes m and effective label-encoding lengths λ_u. For Bic-DC, λ_u is the PRF-masked label-encoding length used in prefix-token matching; for the two-party baselines, it is the label length processed by the underlying data-cleaning functionality. Thus, the comparison measures the cost of scaling privacy-preserving data cleaning beyond two parties under matched effective label lengths.

The results show that Bic-DC achieves lower runtime and communication in most settings, despite supporting multiple participants. For $\lambda_u \in \{32, 40, 50\}$, Bic-DC with $n = 4$ achieves the best cost across almost all settings, showing the benefit of bicentric OKVS-based resolution over repeated pairwise executions. For example, at $m = 2^{19}$ and $\lambda_u = 32$,Bic-DC with $n = 4$ requires 404.87 s and 455.17 MB, compared with 1310.46 s and 2068.65 MB for KKRT-PDC$_2$, and 1786.93 s and 1568.67 MB for VOLE-PDC$_2$. When $\lambda_u = 64$, the prefix-token expansion cost becomes more visible, but Bic-DC remains substantially more communication-efficient, especially under bandwidth-constrained networks.

D. Comparison With MPSI-Style Baselines

To isolate the cost of the underlying bicentric matching core, we disable the label-conflict detection module of Bic-DC and compare it with MPSI, a state-of-the-art MPSI protocol [35], and MPSA [13], a representative outsourced/cloud-assisted MPSI protocol. Table IV reports runtime under three bandwidth settings and total communication, with highlighted cells indicating the best result for each metric.

The results show that Bic-DC's intersection core is competitive with intersection-only MPSI protocols. MPSI is often faster under 1 Gbps because it is optimized for plain intersection, whereas Bic-DC becomes more advantageous under bandwidth-constrained settings due to its compact bicentric communication. For example, when $m = 2^{19}$ and $n = 20$, Bic-DC takes 186.23 s under 10 Mbps, compared with 199.65 s for MPSI and 244.63 s for MPSA, while also reducing communication compared with MPSI, 193.12 MB versus 221.21 MB. Thus, the full Bic-DC protocol adds label-conflict detection and avoids full-intersection disclosure on top of an efficient MPSI-level matching core.

VIII. CONCLUSION

In this paper, we propose Bic-DC, the first scalable protocol for privacy-preserving multi-party data cleaning. Bic-DC introduces a bicentric design that reduces the multi-party inconsistency-detection task to an efficient two-party setting, thereby eliminating quadratic communication and costly public-key operations. To provide task-specific disclosure, we further develop a lightweight prefix-flipping scheme that reveals

only mislabeled records under the explicitly modeled leakage profile, while concealing consistent and non-overlapping records. Experiments show that Bic-DC consistently outperforms two-party baselines and scales efficiently to many participants. This confirms its practicality for real-world collaborative data cleaning while providing a foundation for future extensions. Looking forward, an important direction for future research is extending Bic-DC to stronger adversarial models and exploring its applicability in real-world collaborative environments such as healthcare, finance, and cybersecurity, where both scalability and minimal disclosure are critical.

REFERENCES

[1] S. E. Whang, Y. Roh, H. Song, and J.-G. Lee, "Data collection and quality challenges in deep learning: A data-centric AI perspective," *VLDB J.*, vol. 23, pp. 791–813, 2023.

[2] A. L'heureux, K. Grolinger, H. F. Elyamany, and M. A. Capretz, "Machine learning with Big Data: Challenges and approaches," *IEEE Access*, vol. 5, pp. 7776–7797, 2017.

[3] J. M. Hellerstein, "Quantitative data cleaning for large databases," UC Berkeley, 2013.

[4] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A benchmark for joint data cleaning and machine learning [experiments and analysis]," 2019, *arXiv:1904.09483*.

[5] I. F. Ilyas and X. Chu, *Data Cleaning*. San Rafael, CA, USA: Morgan & Claypool, 2019.

[6] T. Rozario, T. Long, M. Chen, W. Lu, and S. Jiang, "Towards automated patient data cleaning using deep learning: A feasibility study on the standardization of organ labeling," 2017, *arXiv:1801.00096*.

[7] Y. Chen, C. Zhao, Y. Xu, C. Nie, and Y. Zhang, "Year-over-year developments in financial fraud detection via deep learning: A systematic literature review," 2025, *arXiv:2502.00201*.

[8] E.-O. Blass and F. Kerschbaum, "Private collaborative data cleaning via non-equi PSI," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 1419–1434.

[9] L. Kissner and D. Song, "Privacy-preserving set operations," in *Proc. Annu. Int. Cryptol. Conf.*, 2005, pp. 241–257.

[10] C. Hazay and M. Venkitasubramaniam, "Scalable multi-party private set-intersection," in *Proc. IACR Int. Workshop Public Key Cryptography*, 2017, pp. 175–203.

[11] M. J. Freedman, K. Nissim, and B. Pinkas, "Efficient private matching and set intersection," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 2004, pp. 1–19.

[12] N. Chandran, N. Dasgupta, D. Gupta, S. L. B. Obbattu, S. Sekar, and A. Shah, "Efficient linear multiparty PSI and extensions to circuit/quorum PSI," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1182–1204.

[13] Y. Xi, Y. Guo, S. Xu, C. Cai, and X. Jia, "Private sample alignment for vertical federated learning: An efficient and reliable realization," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3834–3848, Mar. 2025.

[14] A. Bay, Z. Erkin, J.-H. Hoepman, S. Samardjiska, and J. Vos, "Practical multi-party private set intersection protocols," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1–15, Oct. 2022.

[15] T. Reinhold, P. Kuehn, D. Günther, T. Schneider, and C. Reuter, "ExTRUST: Reducing exploit stockpiles with a privacy-preserving depletion system for inter-state relationships," *IEEE Trans. Technol. Soc.*, vol. 4, no. 2, pp. 158–170, Jun. 2023.

[16] O. Nevo, N. Trieu, and A. Yanai, "Simple, fast malicious multiparty private set intersection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1151–1165.

[17] R. Inbar, E. Omri, and B. Pinkas, "Efficient scalable multiparty private set-intersection via garbled bloom filters," in *Proc. Int. Conf. Secur. Cryptography Netw.*, 2018, pp. 235–252.

[18] E. Zhang, F.-H. Liu, Q. Lai, G. Jin, and Y. Li, "Efficient multi-party private set intersection against malicious adversaries," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, 2019, pp. 93–104.

[19] I.-S. U. Arbitrary, "Practical multi-party private set intersection cardinality and intersection-sum under arbitrary collusion," in *Proc. Int. Conf. Inf. Secur. Cryptol.*, 2023, pp. 169–191.

[20] S. K. Debnath, P. Stănică, N. Kundu, and T. Choudhury, "Secure and efficient multiparty private set intersection cardinality," *Adv. Math. Commun.*, vol. 15, pp. 365–386, 2021.

[21] R.-H. Shi, "Quantum multiparty privacy set intersection cardinality," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 4, pp. 1203–1207, Apr. 2021.

[22] S. Badrinarayanan, P. Miao, S. Raghuraman, and P. Rindal, "Multi-party threshold private set intersection with sublinear communication," in *Proc. IACR Int. Workshop Public Key Cryptography*, 2021, pp. 349–379.

[23] P. Branco, N. Döttling, and S. Pu, "Multiparty Cardinality Testing for threshold private intersection," in *Proc. IACR Int. Workshop Public Key Cryptography*, 2021, pp. 32–60.

[24] F.-H. Liu, E. Zhang, and L. Qin, "Efficient multiparty probabilistic threshold private set intersection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2023, pp. 2188–2201.

[25] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," *Proc. VLDB Endowment*, vol. 10, pp. 1190–1201, 2017.

[26] X. Chu, I. F. Ilyas, S. Krishnan, and J. Wang, "Data cleaning: Overview and emerging challenges," in *Proc. Int. Conf. Manage. Data*, 2016, pp. 2201–2206.

[27] S. Schelter, Y. He, J. Khilnani, and J. Stoyanovich, "FairPrep: Promoting data to a first-class citizen in studies on fairness-enhancing interventions," 2019, *arXiv:1911.12587*.

[28] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," *Proc. VLDB Endowment*, vol. 9, pp. 948–959, 2016.

[29] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "CleanML: A study for evaluating the impact of data cleaning on ML classification tasks," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 13–24.

[30] S. Guha, F. A. Khan, J. Stoyanovich, and S. Schelter, "Automated data cleaning can hurt fairness in machine learning-based decision making," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 12, pp. 7368–7379, Dec. 2024.

[31] G. Simonini et al., "Entity resolution on-demand," *Proc. VLDB Endowment*, vol. 15, pp. 1506–1518, 2022.

[32] M. T. Islam, A. Fariha, A. Meliou, and B. Salimi, "Through the data management lens: Experimental analysis and evaluation of fair classification," in *Proc. Int. Conf. Manage. Data*, 2022, pp. 232–246.

[33] M. Dolatshah, "Cleaning crowdsourced labels using oracles for statistical classification," *Proc. VLDB Endowment*, vol. 12, pp. 376–389, 2018.

[34] Y. Liang et al., "Collaborative edge server placement for maximizing QoS with distributed data cleaning," *IEEE Trans. Services Comput.*, vol. 18, no. 3, pp. 1321–1335, May/Jun. 2025.

[35] Y. Gao et al., "Efficient scalable multi-party private set intersection (-variants) from bicentric zero-sharing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 4137–4151.

[36] V. Kolesnikov, N. Matania, B. Pinkas, M. Rosulek, and N. Trieu, "Practical multi-party private set intersection from symmetric-key techniques," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1257–1272.

[37] J. H. Cheon, S. Jarecki, and J. H. Seo, "Multi-party privacy-preserving set intersection with quasi-linear complexity," *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, vol. 95, pp. 1366–1378, 2012.

[38] A. Kavousi, J. Mohajeri, and M. Salmasizadeh, "Efficient scalable multi-party private set intersection using oblivious PRF," in *Proc. 17th Int. Workshop Secur. Trust Manage.*, 2021, pp. 81–99.

[39] P. Rindal and P. Schoppmann, "VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2021, pp. 901–930.

[40] O. Goldreich, S. Goldwasser, and S. Micali, "How to construct random functions," *J. ACM*, vol. 33, pp. 792–807, 1986.

[41] O. Goldreich, *Foundations of Cryptography*, vol. 2. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[42] Y. Lindell, "How to simulate it–A tutorial on the simulation proof technique," in *Tutorials on the Foundations of Cryptography : Dedicated to Oded Goldreich*. Berlin, Germany: Springer, 2017.

[43] L. Zhang, F. Qiu, F. Hao, and H. Kan, "1-round distributed key generation with efficient reconstruction using decentralized CP-ABE," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 894–907, Feb. 2022.

[44] Y. Zhang, M. Wang, Y. Guo, and F. Guo, "Towards dynamic and reliable private key management for hierarchical access structure in decentralized storage," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 3371–3380.

[45] S. Raghuraman and P. Rindal, "Blazing fast PSI from improved OKVS and subfield VOLE," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2505–2517.

[46] G. Garimella, B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "Oblivious key-value stores and amplification for private set intersection," in *Proc. Annu. Int. Cryptol. Conf.*, 2021, pp. 395–425.
[47] P. Rindal and L. Roy, "LibOTe: An efficient, portable, and easy to use oblivious transfer library," 2016.

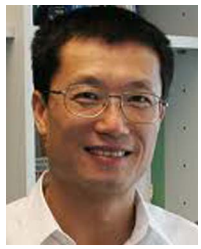
Yuxin Xi received the MS degree from Beijing Normal University. She is currently working toward the PhD degree with the School of Artificial Intelligence, Beijing Normal University, Beijing, China. Her research interests include privacy-preserving computation, cloud computing security, network security, and secure data collaboration. She has worked on privacy-preserving sample alignment, private set computation, and collaborative data cleaning.

Yu Guo (Senior Member, IEEE) received the BE degree in software engineering from Northeastern University, in 2013, and the MSc degree in electronic commerce and the PhD degree in computer science from the City University of Hong Kong, in 2014 and 2019. He is currently an associate professor with the School of Artificial Intelligence, Beijing Normal University. He has also been a postdoctoral and research fellow with the City University of Hong Kong. He is a co-recipient of the Best Paper Award of IEEE ICDCS 2020 and MMM 2016. His research interests include cloud computing security, network security, privacy-preserving data processing, and blockchain technology.

Chen Zhang received the BE degree in network engineering from the Harbin University of Science and Technology, in 2017, the ME degree in computer science and technology from the Harbin Institute of Technology, Shenzhen, in 2019, and the PhD degree in computer science from the City University of Hong Kong, in 2023. She is currently an assistant professor with the Department of Computer Science, Hang Seng University of Hong Kong. Her research interests include cloud computing security, mobile edge computing, federated learning, and blockchain.

Purong Zhang received the BS degree in artificial intelligence from Beijing Normal University, in 2025. She is currently working toward the master's degree with the School of Artificial Intelligence, Beijing Normal University, Beijing, China. Her research interests include network obfuscation, cloud storage security, hardware security, and privacy-preserving data processing.

Hai Liu received the BSc and MSc degrees in applied mathematics from the South China University of Technology, and the PhD degree in computer science from the City University of Hong Kong. He is a professor with the Department of Computer Science, Hang Seng University of Hong Kong (HSUHK). Before joining HSUHK, he held several academic posts with the University of Ottawa and Hong Kong Baptist University. His research interests include wireless networking, cloud computing, artificial intelligence, and algorithm design and analysis.



Xiaohua Jia (Fellow, IEEE) received the BSc and MEng degrees from the University of Science and Technology of China, in 1984 and 1987, respectively, and the DSc degree in information science from the University of Tokyo, in 1991. He is an adjunct with the Harbin Institute of Technology (Shenzhen) while performing this work. He is currently the chair professor with the Department of Computer Science, City University of Hong Kong. His research interests include cloud computing and distributed systems, computer networks, wireless sensor networks, and mobile wireless networks. He is an editor of the *IEEE Transactions on Parallel and Distributed Systems (2006-2009), Wireless Networks, Journal of World Wide Web, Journal of Combinatorial Optimization, etc.* He is the general chair of ACM MobiHoc 2008, TPC co-chair of IEEE MASS 2009, area-chair of IEEE INFOCOM 2010, TPC co-chair of IEEE GlobeCom 2010-Ad Hoc and Sensor Networking Symposium, and Panel co-chair of IEEE INFOCOM 2011. He is a fellow of the IEEE Computer Society. He is an ACM fellow.